



Python函数



目录

Contents

- ◆ 函数介绍
- ◆ 函数说明文档
- ◆ 函数的参数
- ◆ 函数的返回值
- ◆ 函数的嵌套调用
- ◆ 函数的应用
- ◆ 变量的作用域
- ◆ 多函数程序执行流程

学习目标

Learning Objectives

1. 知道函数的作用
2. 知道函数的使用步骤



函数介绍

什么是函数

函数:是一个被命名的、独立的、完成特定功能的代码段，其可能给调用它的程序一个返回值

通俗的说函数就是把一段可以实现某种功能的代码封装起来，想要使用这个功能就可以直接调用函数

被命名的：在Python中，大部分函数都是有名函数

完成特定功能的代码段：函数的功能要专一，专门为了完成某个功能而定义

返回值：当函数执行完毕后，其可能会返回一个值给函数的调用处

函数的主要作用：

- ① 模块化编程
- ② 代码重用

什么是函数

- 函数中的几个重要概念：
 - 函数
 - 参数
 - 返回值
- 如果把函数比作一个机器，那么参数就是源材料，返回值就是最终产品，同时根据参数的不同最终产品也会有不同



举例说明

用户到ATM机取钱：

1. 输入密码后显示“选择功能”界面
2. 查询余额后显示“选择功能”界面
3. 取2000钱后显示“选择功能”界面



问：1, 2, 3操作之后都会显示**选择功能页面**，那么显示“选择功能”界面**需要重复输出给用户**，怎么实现呢？

答：使用函数把“选择功能”的相关**代码封装**起来，当需要使用的时候直接调用函数，这样就可以**实现代码的复用了**

函数的定义

函数的定义：

```
def 函数名(参数):  
    函数体  
    ...  
    return 返回值
```

函数的调用：

函数名(参数)

注意事项

- ① 不同的需求，参数可有可无
- ② 函数必须先定义后使用

案例

需求：复现ATM取钱功能

搭建整体框架

```
print('密码正确登录成功')
# 显示"选择功能"界面
print('查询余额完毕')
# 显示"选择功能"界面
print('取了2000元钱')
# 显示"选择功能"界面
```

确定“选择功能”界面内容

```
print('-----请选择功能-----')
print('查询余额')
print('存款')
print('取款')
print('-----请选择功能-----')
```

封装“选择功能”

```
# 封装ATM机功能选项 -- 定义函数
def select_func():
    print('-----请选择功能-----')
    print('查询余额')
    print('存款')
    print('取款')
    print('-----请选择功能-----')
```

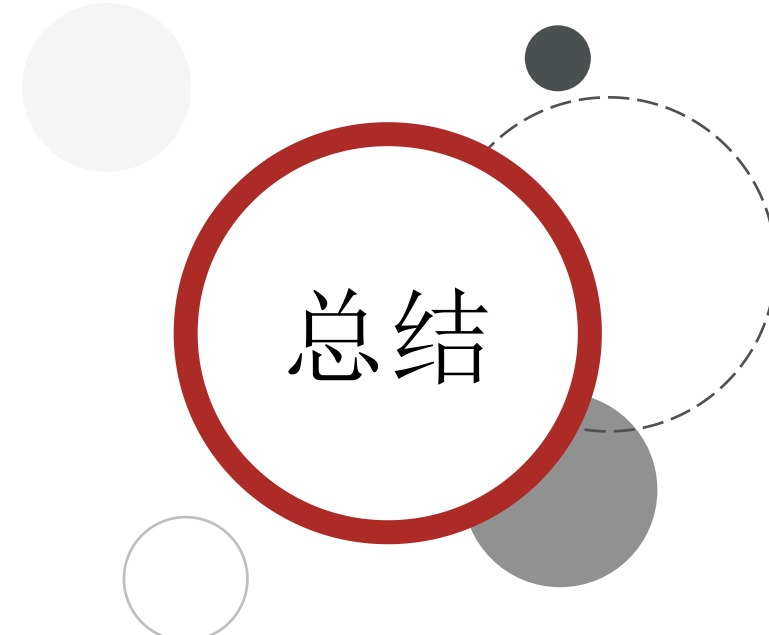
调用函数

```
print('密码正确登录成功')
# 显示"选择功能"界面
-- 调用函数
select_func()
print('查询余额完毕')
# 显示"选择功能"界面
-- 调用函数
select_func()
print('取了2000元钱')
# 显示"选择功能"界面
-- 调用函数
select_func()
```

练一练

要求：定义一个函数，能够输出自己的姓名和年龄，并且调用这个函数让它执行

- 使用def定义函数
- 编写完函数之后，通过 函数名() 进行调用



总结

1. 函数的作用：对代码进行封装，进而提高代码的复用率
2. 函数使用步骤：
 - 先定义函数
 - 后调用函数



目录

Contents

- ◆ 函数介绍
- ◆ 函数说明文档
- ◆ 函数的参数
- ◆ 函数的返回值
- ◆ 函数的嵌套调用
- ◆ 函数的应用
- ◆ 变量的作用域
- ◆ 多函数程序执行流程

学习目标

Learning Objectives

1. 知道什么是函数说明文档



函数的说明文档

什么是函数的说明文档

思考：定义一个函数后，程序员如何书写程序能够快速提示这个函数的作用？

答：注释

思考：如果代码多，我们是不是需要在很多代码中找到这个函数定义的位置才能看到注释？如果想更方便的查看函数的作用怎么办？

答：函数的说明文档（函数的说明文档也叫函数的文档说明）。

函数的说明文档的定义

☆ 定义函数的说明文档

```
def 函数名(参数):  
    """ 说明文档的位置 """  
    代码  
    .....
```

☆ 查看函数的说明文档

```
help(函数名)
```

案例演示:

```
def test(a,b):  
    "用来完成对2个数求和"  
    print( "%d" % (a+b))  
  
test(11,22)  
➔ 33
```




目录

Contents

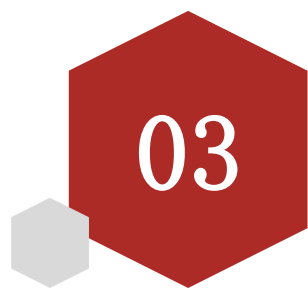
- ◆ 函数介绍
- ◆ 函数说明文档
- ◆ 函数的参数
- ◆ 函数的返回值
- ◆ 函数的嵌套调用
- ◆ 函数的应用
- ◆ 变量的作用域
- ◆ 多函数程序执行流程



学习目标

Learning Objectives

1. 知道函数参数的作用
2. 知道形参和实参的区别



函数的参数

函数参数的作用

函数的参数可以让函数使用更加灵活

思考：一个函数完成两个数1和2的加法运算，如何书写程序？

```
# 定义函数
def add_num1():
    result = 1 + 2
    print(result)

# 调用函数
add_num1()
```

思考：上述add_num1函数只能完成数字1和2的加法运算，如果想要这个函数变得更灵活，可以计算任何用户指定的两个数字的和，如何书写程序？

函数参数的作用

分析：用户要在调用函数的时候指定具体数字，那么在定义函数的时候就需要接收用户指定的数字

函数调用时指定的数字和定义函数时候接收的数字即是函数的参数

```
# 定义函数时同时定义了接收用户数据的参数a和b，a和b是形参
```

```
def add_num2(a, b):  
    result = a + b  
    print(result)
```

```
# 调用函数时传入了真实的数据10 和 20，真实数据为实参
```

```
add_num2(10, 20)
```

带有参数的函数执行流程

➡ #定义接收2个参数的函数
def add2num(a , b):
 c = a+b
 print c

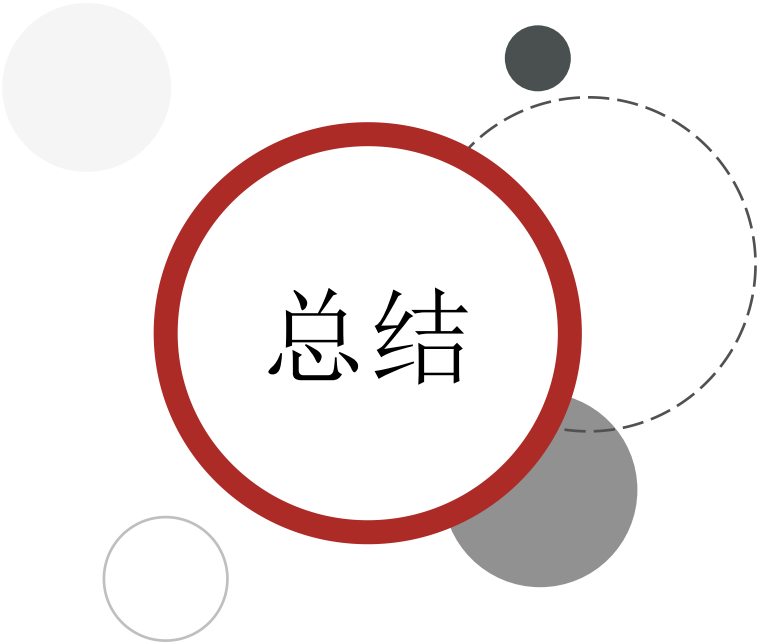
#调用带有参数的函数
add2num(110, 22)

终端

练一练

要求：定义一个函数，完成前2个数完成加法运算，然后对第3个数，进行减法；然后调用这个函数

- 使用def定义函数，要注意有3个参数
- 调用的时候，这个函数定义时有几个参数，那么就需要传递几个参数



总结

1. 定义时小括号中的参数，用来接收参数用的，称为 “形参”
2. 调用时小括号中的参数，用来传递给函数用的，称为 “实参”



目录

Contents

- ◆ 函数介绍
- ◆ 函数说明文档
- ◆ 函数的参数
- ◆ 函数的返回值
- ◆ 函数的嵌套调用
- ◆ 函数的应用
- ◆ 变量的作用域
- ◆ 多函数程序执行流程

学习目标

Learning Objectives

1. 知道函数返回值的作用



函数的返回值

返回值的介绍

生活中的返回值:

我给儿子10块钱，让他给我买包烟。这个例子中，10块钱是我给儿子的，就相当于调用函数时传递到参数，那么不管这个买烟的过程是什么样的，最终的目标就是得到这包烟，那么这包烟就是返回值。

开发中的返回值:

定义一个函数去计算数字计算，那么不管数字的计算过程是什么样的，最终这个函数的目的是把计算的结果返回给函数的调用者，计算的结果就是返回值。

- 综上所述:

所谓“返回值”，就是程序中函数完成一件事情后，最后给调用者的结果



返回值return

- 想要在函数中把结果返回给调用者，需要在函数中使用return

```
def add2num(a, b):  
    c = a+b  
    return c
```

- 保存函数的返回值

在“买烟”的例子中，最后儿子给你烟时，你一定是要从儿子手中接过来的。程序也是如此，如果一个函数返回了一个数据，那么想要用这个数据，那么就**需要保存**

```
#定义函数  
def add2num(a, b):  
    return a+b  
  
#调用函数，顺便保存函数的返回值  
result = add2num(100, 98)  
#因为result已经保存了add2num的返回值，所以接下来就可以使用了  
print(result)
```



目录

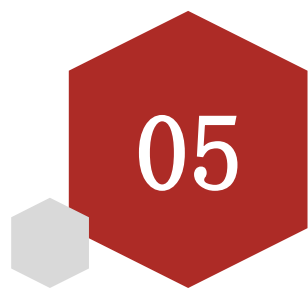
Contents

- ◆ 函数介绍
- ◆ 函数说明文档
- ◆ 函数的参数
- ◆ 函数的返回值
- ◆ 函数的嵌套调用
- ◆ 函数的应用
- ◆ 变量的作用域
- ◆ 多函数程序执行流程

学习目标

Learning Objectives

1. 知道函数如何嵌套调用



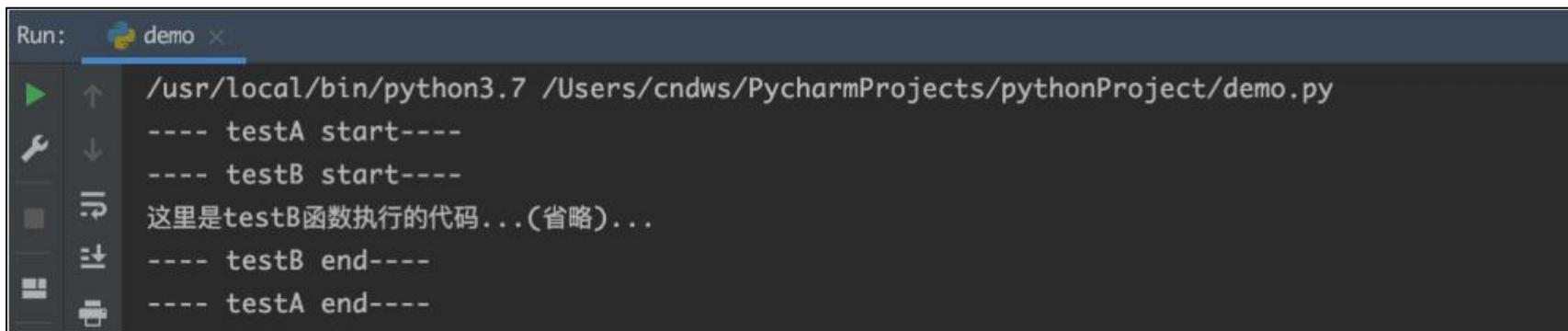
函数的嵌套调用

什么是函数的嵌套

所谓函数嵌套调用指的是一个函数里面又调用了另外一个函数

```
def testB():  
    print('---- testB start----')  
    print('这里是testB函数执行的代码...(省略)...')  
    print('---- testB end----')  
  
def testA():  
    print('---- testA start----')  
    testB()  
    print('---- testA end----')  
  
testA()
```

执行效果:



```
Run: demo x  
/usr/local/bin/python3.7 /Users/cndws/PycharmProjects/pythonProject/demo.py  
---- testA start----  
---- testB start----  
这里是testB函数执行的代码...(省略)...  
---- testB end----  
---- testA end----
```

执行过程



如果函数A中，调用了另外一个函数B，那么先把函数B中的任务都执行完毕之后才会回到上次 函数A执行的位置



目录

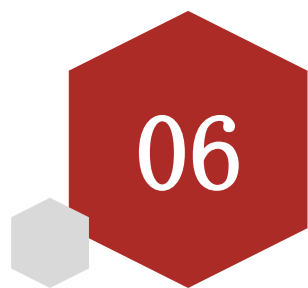
Contents

- ◆ 函数介绍
- ◆ 函数说明文档
- ◆ 函数的参数
- ◆ 函数的返回值
- ◆ 函数的嵌套调用
- ◆ 函数的应用
- ◆ 变量的作用域
- ◆ 多函数程序执行流程

学习目标

Learning Objectives

1. 理解函数的嵌套调用
2. 理解程序设计的思路, 复杂问题分解为简单问题



函数应用案例

打印图形

☆ 打印一条虚线

```
def print_line():  
    print('-' * 20)
```

```
print_line()
```

☆ 打印多条横线

```
def print_lines(num):  
    i = 0  
    while i < num:  
        print_line()  
        i += 1
```

```
print_lines(5)
```

函数计算

☆ 求三个数的和

```
def sum_num(a, b, c):  
    return a + b + c  
  
result = sum_num(1, 2, 3)  
print(result) # 6
```

☆ 求三个数的平均值

```
def average_num(a, b, c):  
    sumResult = sum_num(a, b, c)  
    return sumResult / 3  
  
result = average_num(1, 2, 3)  
print(result) # 2.0
```



目录

Contents

- ◆ 函数介绍
- ◆ 函数说明文档
- ◆ 函数的参数
- ◆ 函数的返回值
- ◆ 函数的嵌套调用
- ◆ 函数的应用
- ◆ 变量的作用域
- ◆ 多函数程序执行流程

学习目标

Learning Objectives

1. 知道什么是局部变量
2. 知道什么是全局变量



变量的作用域

局部变量

变量**作用域**指的是变量的作用范围（变量在哪里可用，在哪里不可用）

主要分为两类：**局部变量和全局变量**

所谓局部变量是**定义在函数体内部的变量，即只在函数体内部生效**

```
def testA():  
    a = 100  
    print(a)  
  
testA()      # 100  
print(a)     # 报错: name 'a' is not defined
```

变量a是定义在`testA`函数内部的变量，在函数外部访问则立即报错.

局部变量的作用：在函数体内部，临时保存数据，即当函数调用完成后，则销毁局部变量

全局变量

所谓全局变量，指的是在函数体内、外都能生效的变量

思考：如果有一个数据，在函数A和函数B中都要使用，该怎么办？

答：将这个数据存储在一个全局变量里面

```
# 定义全局变量a
a = 100

def testA():
    print(a)    # 访问全局变量a，并打印变量a存储的数据

def testB():
    print(a)    # 访问全局变量a，并打印变量a存储的数据

testA()    # 100
testB()    # 100
```

global关键字

思考：`testB`函数需要修改变量a的值为200，如何修改程序？

```
a = 100

def testA():
    print(a)

def testB():
    a = 200
    print(a)

testA()      # 100
testB()      # 200
print(f'全局变量a = {a}')  # 全局变量a = 100
```

`testB`函数内部的 `a = 200` 是定义了一个局部变量

global关键字

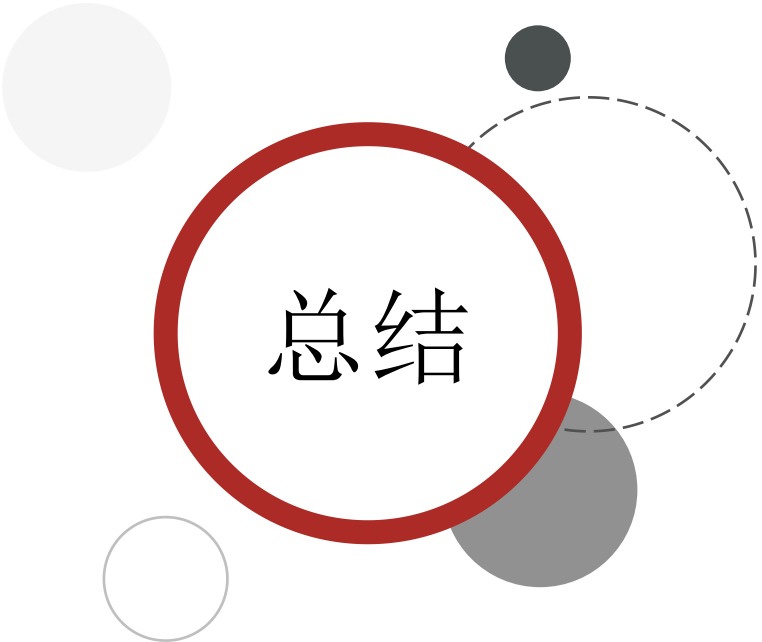
☆ 使用 `global`关键字 可以在函数内部声明变量为全局变量，如下所示

```
a = 100

def testA():
    print(a)

def testB():
    # global 关键字声明a是全局变量
    global a
    a = 200
    print(a)

testA()    # 100
testB()    # 200
print(f '全局变量a = {a} ')    # 全局变量a = 200
```



总结

1. 局部变量：就是在函数内部定义的变量

- 其作用范围是这个函数内部，即只能在这个函数中使用，在函数的外部是不能使用的
- 因为其作用范围只是在自己的函数内部，所以不同的函数可以定义相同名字的局部变量（打个比方，把你、我是当做成函数，把局部变量理解为每个人手里的手机，你可有个iPhone8，我当然也可以有个iPhone8了， 互不相关）
- 局部变量的作用，为了临时保存数据需要在函数中定义变量来进行存储

2. 全局变量：如果一个变量，既能在一个函数中使用，也能在其他的函数中使用，这样的变量就是全局变量，这样的变量一般在函数外部定义

- 在函数外边定义的变量叫做全局变量
- 全局变量能够在所有的函数中进行访问



目录

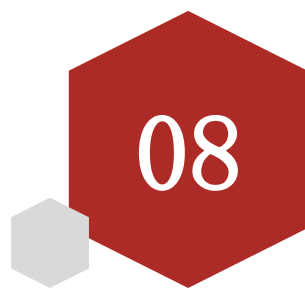
Contents

- ◆ 函数介绍
- ◆ 函数说明文档
- ◆ 函数的参数
- ◆ 函数的返回值
- ◆ 函数的嵌套调用
- ◆ 函数的应用
- ◆ 变量的作用域
- ◆ 多函数程序执行流程

学习目标

Learning Objectives

1. 知道多个函数之间怎么执行



多函数程序执行流程

多函数程序执行流程

一般在实际开发过程中，一个程序往往由多个函数组成，并且多个函数共享某些数据，如下所示：

☆ 共用全局变量：

```
# 1. 定义全局变量
glo_num = 0

def test1():
    global glo_num    # 修改全局变量
    glo_num = 100

def test2():
    # 调用test1函数中修改后的全局变量
    print(glo_num)

# 2. 调用test1函数，执行函数内部代码：声明和修改全局变量
test1()
# 3. 调用test2函数，执行函数内部代码：打印
test2()    # 100
```

多函数程序执行流程

☆ 把函数返回值作为参数传递:

```
def test1():  
    return 50  
  
def test2(num):  
    print(num)  
  
# 1. 保存函数test1的返回值  
result = test1()  
  
# 2. 将函数返回值所在变量作为参数传递到test2函数  
test2(result)    # 50
```



传智教育旗下高端IT教育品牌